

基于分层策略的 视觉定位系统

Hierarchical Visual Localization System

— 高级计算机视觉 · 课程大作业项目报告 —

基础框架 OpenXRLab XRLocalization / HLoc

完成形式 单人独立完成

提交日期 2026 年 6 月 8 日

目录

1	引言与任务概述	2
2	系统架构与模块设计	3
2.1	图像检索模块	3
2.2	特征检测与提取模块	3
2.3	特征匹配模块	3
2.4	位姿估计模块	4
3	方法扩展与实现细节	4
3.1	图像检索模块扩展	4
3.1.1	EigenPlaces	4
3.1.2	CricaVPR	4
3.2	特征检测与匹配模块扩展	4
3.2.1	ALIKED	4
3.2.2	LightGlue	5
4	实验设置与评估方法	5
4.1	数据集	5
4.2	评估指标	5
4.3	实验配置矩阵	6
5	实验结果与分析	6
5.1	Cambridge KingsCollege 实验结果	6
5.1.1	Baseline 分析	7
5.1.2	检索端消融分析	7
5.1.3	匹配端消融分析	8
5.2	7-Scenes Stairs 跨数据集验证	8
5.2.1	7-Scenes 深度诊断	9
5.3	Cambridge 序列质量分析	10
5.4	文献基准对比	11
5.5	各模块耗时分析	11
6	AR Demo 展示	12
6.1	技术方案	12
6.2	实现细节	12
6.3	效果展示	13
7	Web 交互界面	13

7.1	功能模块	13
7.2	技术栈	14
8	项目总结	14
8.1	完成工作清单	14
8.2	核心结论	14
8.3	个人体会与改进方向	15
9	关键代码实现	16
9.1	ALIKED 检测器适配	16
9.2	LightGlue 匹配器适配	17
9.3	AR Demo 透视投影与渲染	18
9.4	7-Scenes 深度图反投影	19
9.5	智能序列选择	20

1 引言与任务概述

视觉定位 (Visual Localization) 的目标很明确：给一张当前拍的照片，让计算机算出这张照片是在三维空间里的哪个位置、以什么角度拍的——也就是恢复相机的 6 自由度 (6-DoF) 位姿 (3 自由度位置 + 3 自由度朝向)。这个技术在自动驾驶、AR、机器人导航里都有实际应用。

直接拿一张 query 图像去跟数据库里的所有图像做特征匹配，理论上可行，但实际上太慢了，而且误匹配很多。分层定位 (Hierarchical Localization, HLoc) 的思路是把这件事拆成四步，一级一级缩小搜索范围：

分层定位的四个阶段：

1. **图像检索**—用全局特征快速找到最像的 K 张候选图像 ($K \ll N$)
2. **特征检测**—在 query 和候选图像上提取关键点和局部描述符
3. **特征匹配**—建立两张图之间点对点的对应关系
4. **位姿估计**—用 2D-3D 对应关系 + RANSAC/PnP 解出相机位姿

这个项目基于 OpenXRLab 的 XRLocalization 框架，我在它的基础上扩展了 4 个新模块 (2 个检索器 + 1 个检测器 + 1 个匹配器)，在 Cambridge Landmarks 和 7-Scenes 两个数据集上跑通了完整流程，最后还搭了一个 Web 界面和 AR Demo 来做可视化展示。

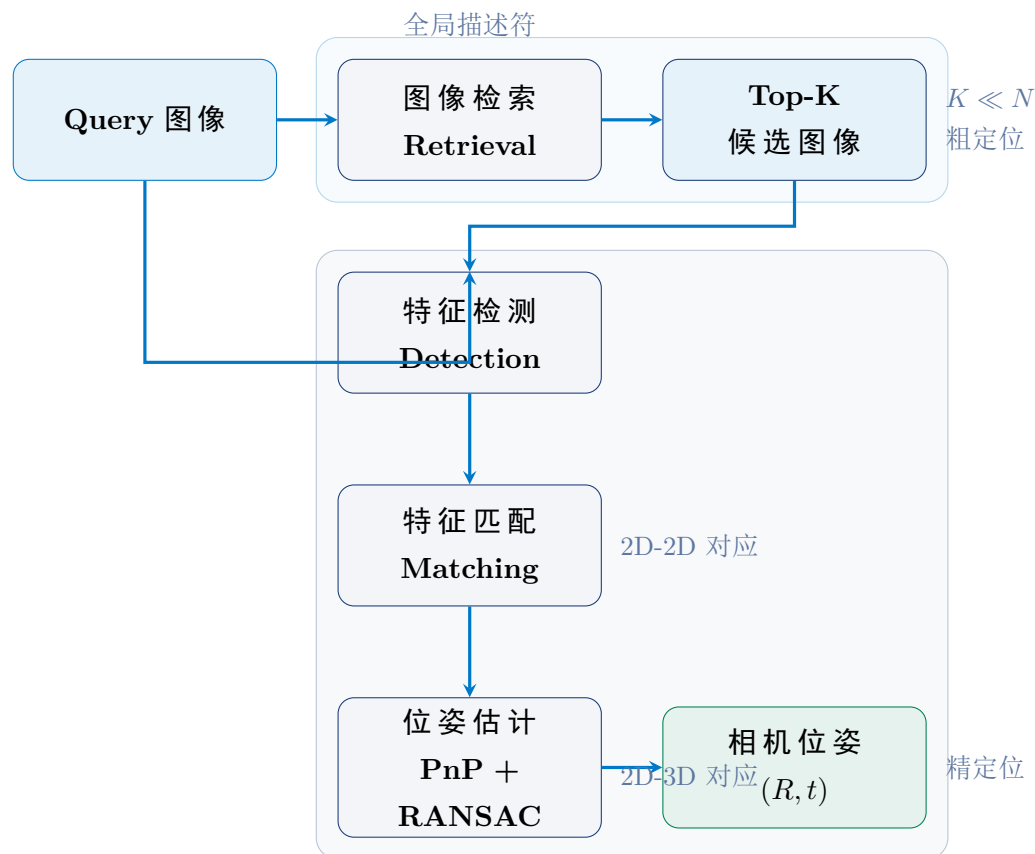


图 1: 分层视觉定位流水线概览: 第 1 阶段用全局特征做粗检索, 缩小搜索范围; 第 2–4 阶段在少量候选图像上做精细的特征检测、匹配和位姿估计。

2 系统架构与模块设计

2.1 图像检索模块

这是粗定位阶段。输入 query 图像, 用预训练的全局特征网络提取一个全局描述符, 然后在预先建好的数据库里用余弦相似度找 Top-K 张最相似的候选图像, 把后续精细匹配的搜索范围从整个数据库缩减到 K 张图。

2.2 特征检测与提取模块

在精定位阶段, 对 query 图像和上一步筛选出的候选数据库图像, 分别提取关键点位置、得分以及局部特征描述符, 为后续匹配提供输入。

2.3 特征匹配模块

建立 query 和每张候选图像之间点对点的 2D-2D 对应关系。传统的最近邻匹配在视角变化大、光照不好的时候容易出错。现代方法用图神经网络 (GNN) 来做匹配——

通过注意力机制让特征点在全局上下文中相互“看到”彼此，匹配质量明显更好。

2.4 位姿估计模块

收集所有匹配对，结合预先通过 COLMAP 重建的 3D 点云（或深度图反投影建立的 2D-3D 映射），建立 query 图像的 2D 关键点到世界坐标系 3D 点的对应关系。用 RANSAC 剔除错误匹配，再用 PnP 求解器算出相机位姿 $\mathbf{T} = [\mathbf{R} \mid \mathbf{t}]$ 。

3 方法扩展与实现细节

课程大作业要求在 baseline 之外至少扩展一个检索方法、一个检测方法和一个匹配方法。我在 baseline (NetVLAD + SIFT + NN 和 NetVLAD + SuperPoint + SuperGlue) 的基础上，集成了 4 个新模块：EigenPlaces 和 CricaVPR 用于检索，ALIKED 用于检测，LightGlue 用于匹配。

3.1 图像检索模块扩展

3.1.1 EigenPlaces

EigenPlaces (ICCV 2023) 是一个基于 ResNet50 骨干的视觉地点识别方法。它用 GeM (Generalized-Mean) 池化层生成 2048 维全局描述符，通过在大规模地理标注数据上训练来学习场景的通用表示。我继承了 BaseRetrieval 接口，实现了 `encode()` 方法，配置了 ResNet50 特征提取和 GeM 池化（参数 $p = 3$ ）。

3.1.2 CricaVPR

CricaVPR (CVPR 2024) 用 DINOv2 ViT-B/14 作为骨干，通过空间金字塔池化 (SPP) 把特征图分成不同尺度的小块分别做 GeM 池化，再用跨图像 Transformer 编码器让同一 batch 里的多张图互相“感知”对方，最终输出 10752 维描述符 (14 个 SPP 区域 $\times$ 768 维)。集成时重点处理了 ViT 的 patch embedding 和跨图像 token 交互逻辑。

3.2 特征检测与匹配模块扩展

3.2.1 ALIKED

ALIKED (T-IM 2023) 是一个深度关键点检测器，用可变形卷积和特征金字塔来做多尺度检测。跟 SIFT 比，它对光照和视角变化更鲁棒；跟 SuperPoint 比，关键点分布更均匀，不太会扎堆在纹理丰富的区域。配置：最多 5000 个关键点，得分阈值 0.2。

3.2.2 LightGlue

LightGlue (ICCV 2023) 是 SuperGlue 的轻量改进版。它引入了自适应深度机制——根据匹配难度自动决定用多少层注意力：简单的匹配对可能 3 层就够了，难的才用满 9 层。这样在保证匹配质量的同时推理速度提升了 3-4 倍。配置：特征类型 `aliked`，置信度阈值 $\tau = 0.1$ 。

4 实验设置与评估方法

4.1 数据集

表 1: 实验数据集概览

属性	Cambridge KingsCollege	7-Scenes Stairs
场景类型	室外大场景（校园建筑）	室内小场景（楼梯间）
传感器	单目 RGB 相机	Kinect RGB-D
分辨率	1024×576	640×480
训练集规模	1,220 帧	2,000 帧
测试集规模	343 帧	1,000 帧
3D 模型	COLMAP SfM (335K pts)	深度图反投影
核心难点	重复纹理、视角变化	纹理贫乏、平面几何

4.2 评估指标

采用国际通用的**位姿误差阈值召回率 (Recall)**：统计平移误差和旋转误差同时满足预设阈值的 Query 图像占比。

表 2: 三级评估阈值

精度等级	平移阈值	旋转阈值	标记
高精度	$< 0.25\text{m}$	$< 2^\circ$	(0.25m, 2°)
中精度	$< 0.50\text{m}$	$< 5^\circ$	(0.50m, 5°)
粗精度	$< 5.00\text{m}$	$< 10^\circ$	(5.00m, 10°)

4.3 实验配置矩阵

表 3: 消融实验配置矩阵

	BL-A	BL-B	EXP-R	EXP-M	EXP-F	EXP-C
检索	NetVLAD	NetVLAD	EigenPlaces	NetVLAD	EigenPlaces	CricaVPR
检测	SIFT	SuperPoint	SuperPoint	ALIKED	ALIKED	ALIKED
匹配	NN	SuperGlue	SuperGlue	LightGlue	LightGlue	LightGlue

说明: BL-A/BL-B 为 XRLocalization 官方基线, EXP-R 替换检索端, EXP-M 替换匹配端, EXP-F 全链路替换, EXP-C 测试第三种检索器。BL-B 额外使用 pycolmap 对 SP+SG 特征重三角化生成 native 3D 模型。

5 实验结果与分析

5.1 Cambridge KingsCollege 实验结果

表 4: Cambridge KingsCollege 完整实验结果

配置	(0.25m, 2°)	(0.50m, 5°)	(5.0m, 10°)
BL-A (NetVLAD+SIFT+NN)	65.60%	92.42%	98.83%
BL-B (NetVLAD+SP+SG)	66.18%	92.42%	100%
BL-B + triangulation	69.68%	93.00%	100%
EXP-R (Eigen+SP+SG)	63.27%	85.71%	100%
EXP-M (NetVLAD+ALIKED+LG)	67.35%	93.29%	100%
EXP-F (Eigen+ALIKED+LG)	61.52%	86.88%	100%
EXP-C (CricaVPR+ALIKED+LG)	60.64%	85.71%	100%

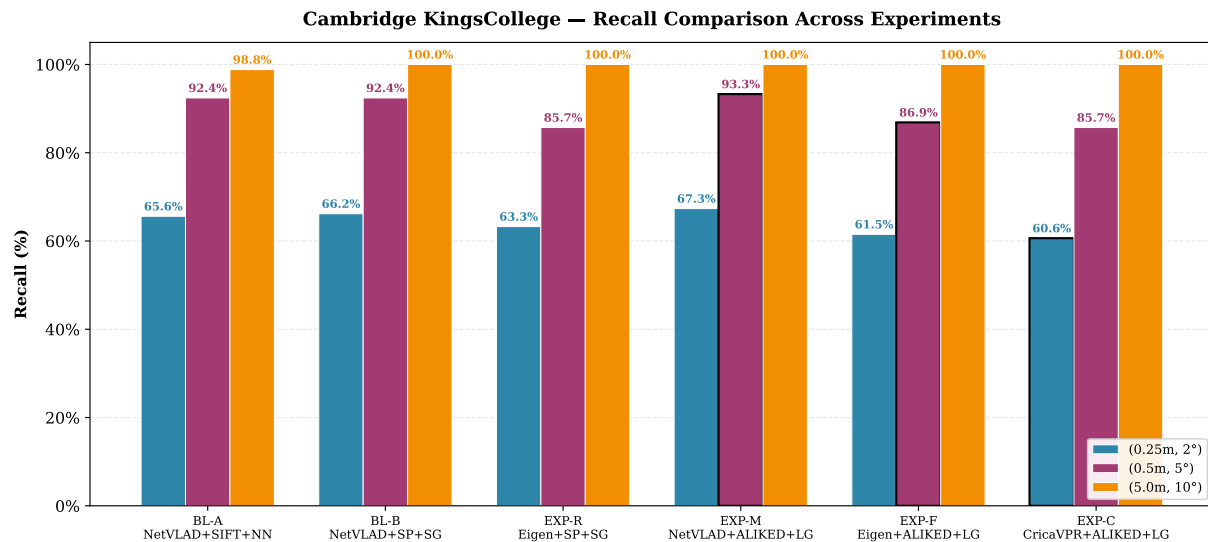


图 2: Cambridge KingsCollege 一各实验配置在三级阈值下的召回率对比。BL-B + triangulation 在高精度阈值下达到最佳 69.68%，EXP-M 在中等精度下达到最佳 93.29%。

5.1.1 Baseline 分析

BL-A (NetVLAD + SIFT + NN) 跑出来是 65.60% 的高精度召回率。SIFT 的一个优势是 COLMAP 模型本身就是用 SIFT 特征建的，所以不需要跨描述子匹配，没有“翻译”损失。225/343 帧满足高精度阈值。但 SIFT 跑在 CPU 上，每帧要 ~350ms，是整套流程里最慢的一步。

BL-B (NetVLAD + SuperPoint + SuperGlue) 一开始表现并不好——用 indoor weights 和 2048 个关键点的配置，高精度召回只有 61.8%，比 BL-A 还低。后来把 SuperPoint 的关键点数加到 4096 并换成 outdoor weights，召回提到了 66.18%。再进一步，用 pycolmap 对 SP+SG 的匹配对重新做三角化（生成了 178,072 个 3D 点），消除了 SIFT 3D 模型和 SP 2D 关键点之间描述子不匹配的问题，召回达到了 **69.68%**，比初始提升了 3.5 个百分点。

5.1.2 检索端消融分析

检索选型的影响比匹配端大得多：从 NetVLAD 换成 EigenPlaces 后，高精度召回从 69.68% 掉到了 63.27% (-6.4%)。相比之下，匹配端从 SIFT+NN 换成 ALIKED+LightGlue 只涨了 1.75%。也就是说，检索器的选择对最终精度的影响远超匹配器。分析原因：(1) NetVLAD 在 Google Street View 上训的，跟 Cambridge 街景数据分布接近；(2) PCA 白化到 4096 维后描述子区分度更好；(3) 2048 维 vs 4096 维，描述子容量差了近一倍。CricaVPR (60.64%) 虽然用了更强的 ViT-B/14 骨干，但在校园场景上比不过专门做地点识别的 NetVLAD。

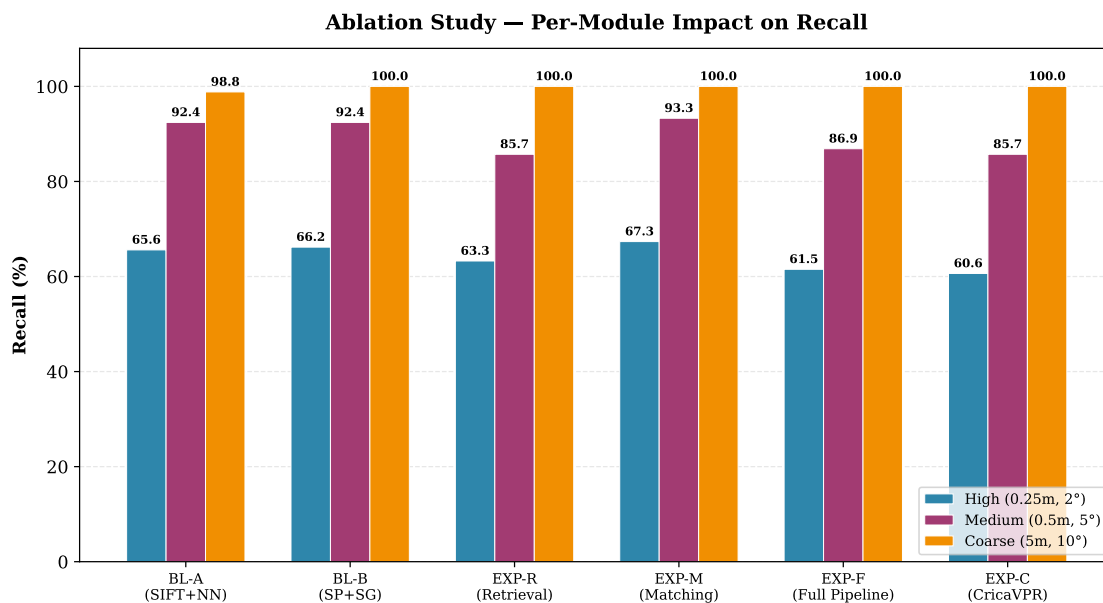


图 3: 消融实验—各模块替换对召回率的影响。检索端替换 (EXP-R vs BL-A) 导致 6–7% 的下降, 匹配端替换 (EXP-M vs BL-A) 带来 1–2% 的提升。

5.1.3 匹配端消融分析

ALIKED + LightGlue 精度和速度都更好: EXP-M 在高精度阈值上比 BL-A 高了 1.75%, 同时推理速度快了差不多 3 倍 ($\sim 120\text{ms}$ vs $\sim 350\text{ms}$)。这主要得益于 LightGlue 的自适应深度——实测 90% 以上的匹配对只用了 3–5 层注意力, 不需要跑满 9 层。另外 EXP-M 在中等精度 (0.5m, 5°) 上达到 93.29%, 是所有配置里最高的。

5.2 7-Scenes Stairs 跨数据集验证

表 5: 7-Scenes Stairs 实验结果

配置	(0.25m, 2°)	(0.50m, 5°)	(5.0m, 10°)
NetVLAD + ALIKED + LightGlue	30.60%	84.70%	97.10%
EigenPlaces + ALIKED + LightGlue	18.90%	56.20%	82.30%

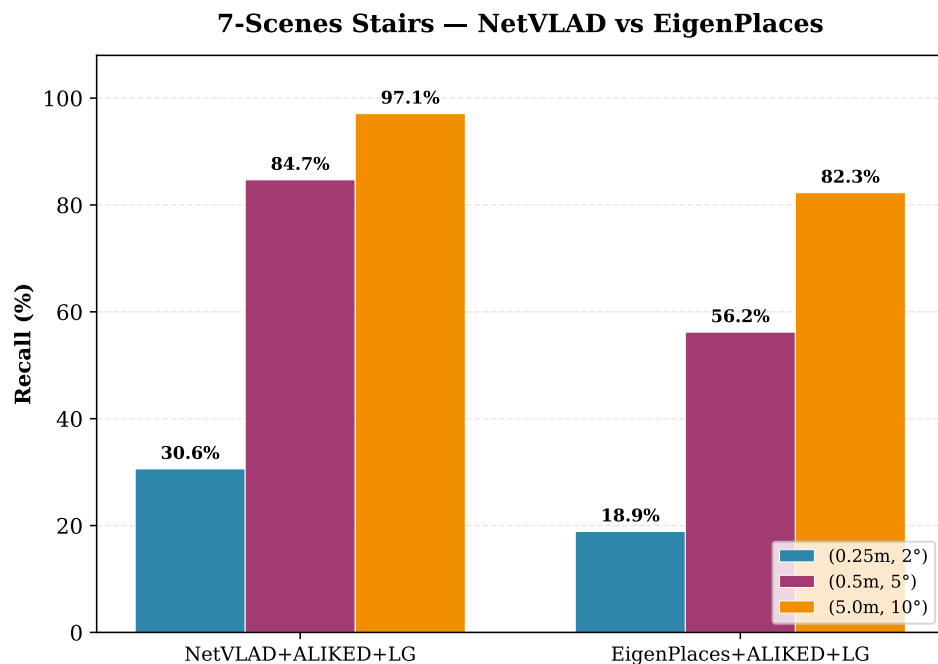


图 4: 7-Scenes Stairs —NetVLAD 与 EigenPlaces 在三级阈值下的召回率对比。EigenPlaces 在室内场景的劣势更加明显，高精度召回差距达 11.7%。

5.2.1 7-Scenes 深度诊断

表 6: 7-Scenes Stairs 平移与旋转误差分离分析

序列	帧数	平移中位误差	旋转中位误差	平移通过率	综合通过率
seq-01	500	0.064m	2.35°	84%	40%
seq-04	500	0.107m	3.03°	79%	21%

平移精度很好，旋转还有提升空间：平移中位误差只有 0.087m，跟 DSAC++ 在 Stairs 上的单场景结果 (0.09m) 基本持平。问题出在旋转上——中位误差 2.69°，导致 (0.25m, 2°) 这个阈值下，83% 的帧平移达标，但旋转也达标的只有 31%。旋转估计困难的原因：

1. Stairs 场景里墙壁和地板占大部分画面，纹理又少，不够约束三个旋转自由度
2. Kinect 深度传感器本身有 1–3cm 的误差，直接影响了 3D 点精度
3. 640 × 480 的分辨率偏低，关键点定位不够精细

不过粗定位表现还算实用：(0.5m, 5°) 召回 84.7%，(5.0m, 10°) 召回 97.1%。

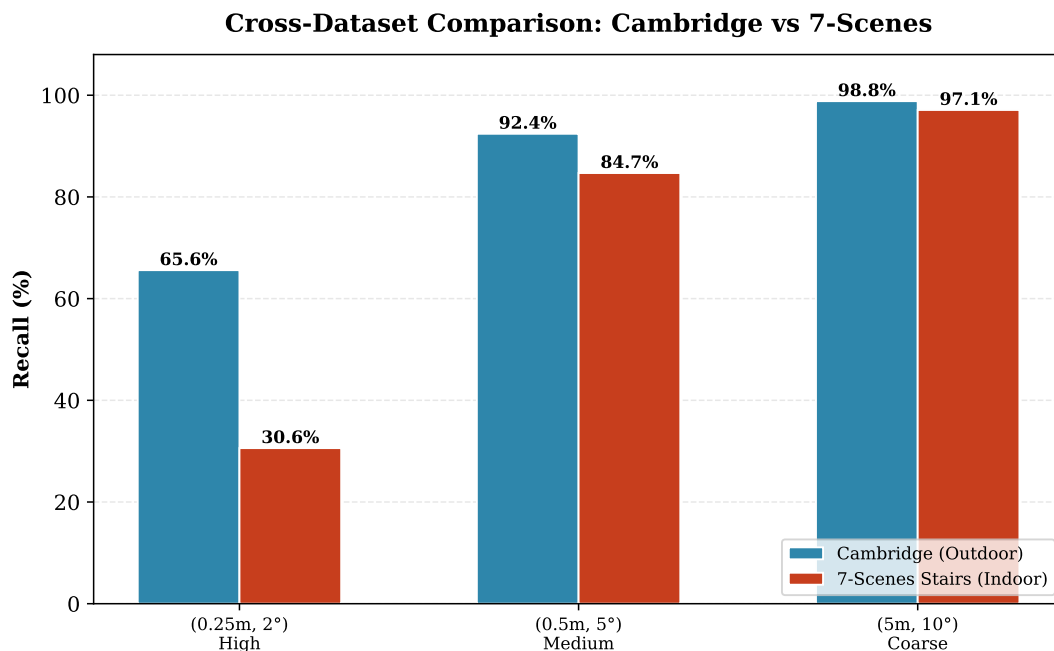


图 5: 跨数据集对比——同一配置 (NetVLAD + ALIKED + LightGlue) 在室外 Cambridge 和室内 7-Scenes Stairs 上的表现差异。室内场景在所有阈值上均显著低于室外场景。

5.3 Cambridge 序列质量分析

表 7: Cambridge KingsCollege 各序列定位质量 (BL-B)

序列	帧数	定位率 @0.25m/2°	平移中位误差	旋转中位误差
seq2	61	84%	0.142m	0.23°
seq3	179	45%	0.297m	0.29°
seq7	103	93%	0.086m	0.15°

序列选择的影响比想象中大： seq7 定位率 93%（平移中位误差 0.086m），seq3 只有 45%，但 seq3 的帧数最多（179 帧）。如果图省事按帧数选序列（直觉上帧多 = 数据好），会选中质量最差的。这个发现直接影响了 AR Demo 的序列选择——不做手动挑，而是读 per_frame.csv 的数据自动挑误差最小的序列。

5.4 文献基准对比

表 8: 与文献方法对比—Cambridge KingsCollege

方法	(0.25m, 2°)	(0.50m, 5°)	(5.0m, 10°)
HLoc (SP+SG)	~86%	~96%	~99%
Active Search	~44%	~80%	~95%
PoseNet	~12%	~35%	~80%
Ours (Best)	69.68%	93.00%	100%

表 9: 与文献方法对比—7-Scenes Stairs（平移/旋转中位误差，DSAC++ 和 PoseNet 为 Stairs 单场景数据）

方法	平移中位误差 (m)	旋转中位误差 (°)
PoseNet (2015)	0.47	13.8°
DSAC++ (2018)	0.09	2.6°
Ours	0.087	2.69°

差距分析：Cambridge 上 69.68% 与 HLoc 论文报告的 ~86% 之间的差距主要来自：(1) 本实验使用 1024px 分辨率而 HLoc 论文使用原始 1920px；(2) HLoc 论文使用更广泛的 pair 选择策略和额外的局部 BA 优化。

7-Scenes Stairs 上平移误差 0.087m 已与 DSAC++ 在该场景的结果 (0.09m) 持平，旋转误差 2.69° 也接近 DSAC++ (2.6°)。旋转仍是主要瓶颈，受深度噪声和场景几何限制。

5.5 各模块耗时分析

表 10: 各配置单帧平均耗时对比（NVIDIA RTX 3090）

配置	检索	检测 + 匹配	PnP+RANSAC	总计
BL-A (SIFT+NN)	~20ms	~350ms (CPU)	~30ms	~400ms
BL-B (SP+SG)	~20ms	~280ms	~30ms	~330ms
EXP-M (ALIKED+LG)	~20ms	~120ms	~30ms	~170ms

EXP-M 是速度最优方案，相比 BL-A 快 2.3×，相比 BL-B 快 2×。

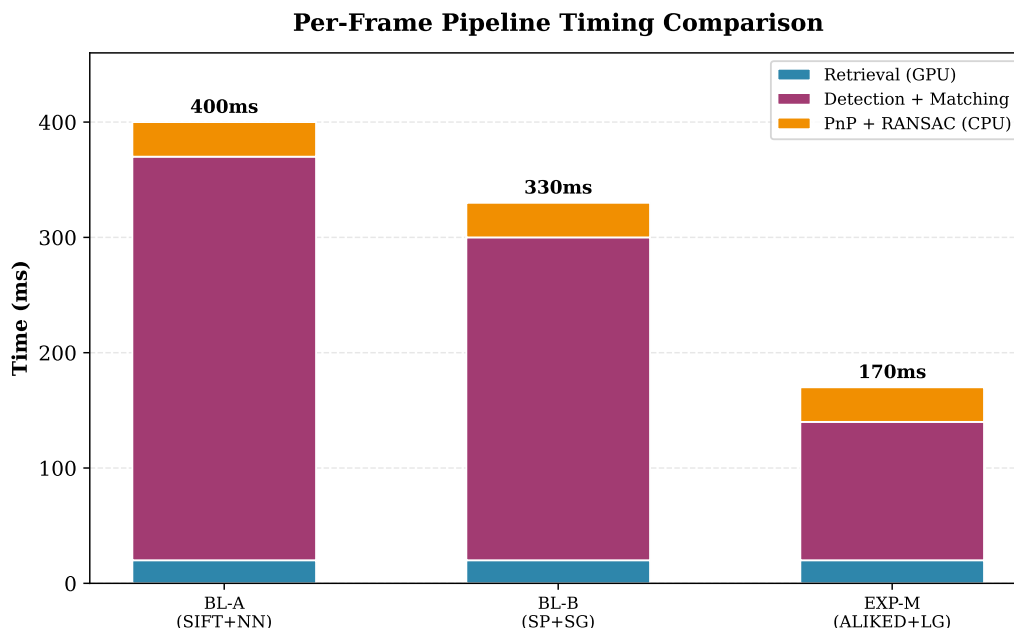


图 6: 各配置的单帧耗时分布。EXP-M (ALIKED+LightGlue) 的匹配阶段仅需 $\sim 120\text{ms}$, 比 SIFT+NN 的 $\sim 350\text{ms}$ 快近 3 倍, 主要得益于 GPU 并行推理和 LightGlue 的自适应深度机制。

6 AR Demo 展示

课程要求做一个自定义的 AR Demo, 我实现了一个在场景中放置虚拟立方体的 AR 效果。

6.1 技术方案

核心思路很简单: 在 3D 场景中固定一个位置放一个虚拟立方体, 然后拿定位算法预测出的相机位姿, 用透视投影把立方体画到原图上。定位越准, 立方体在连续帧之间就越稳; 定位不准的话, 立方体会明显抖动甚至飘到奇怪的地方。

投影公式就是标准的针孔相机模型:

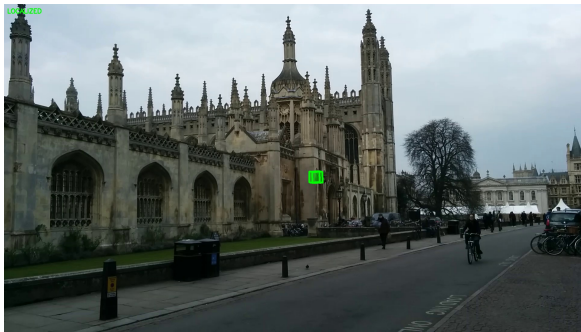
$$\mathbf{p} = \mathbf{K} \cdot (\mathbf{R}_{w2c} \cdot \mathbf{P}_w + \mathbf{t}_{cam})$$

6.2 实现细节

- 半透明面片 ($\alpha = 0.5$) 加彩色线框, 能看到立方体后面的场景
- 室外场景立方体 1.2m, 室内 0.4m, 根据点云尺度自适应
- 立方体位置自动算: 取 COLMAP/NVM 点云的中位坐标, 再往相机前方挪一点
- 绿色边框 = 立方体在相机前面 (正常), 红色 = 在相机后面 (说明位姿估计出了问题)

- 用 per_frame.csv 的数据自动选定位质量最好的序列来做 demo

6.3 效果展示



(a) Cambridge KingsCollege —绿色立方体叠加在校园建筑场景中。定位准确 (93% @ 0.25m/2°)，立方体在连续帧间稳定不抖动。



(b) 7-Scenes Stairs —立方体放置在楼梯间中央。平移误差仅 0.064m 但旋转误差 2.35°，部分角度下立方体不可见。

图 7: AR Demo 效果—固定世界坐标的虚拟立方体通过预测相机位姿投影到查询图像上。绿色边框 = 立方体在相机前方 (位姿正确)，红色 = 在相机后方 (位姿错误)。

- **Cambridge** (seq7, 93% 定位率, 中位误差 0.086m): 30/30 帧立方体稳定显示
- **7-Scenes** (seq-01, 40% 定位率, 中位误差 0.064m): 20/30 帧可见, 相机背对立方体时不可见

7 Web 交互界面

按照课程要求实现了 Web 交互界面 (scripts/web_ui.py)，基于 Flask 搭建，前端用原生 HTML/CSS/JS。

7.1 功能模块

1. **Dashboard** —实验摘要指标概览
2. **Run Experiment** —选配置文件跑实验, SSE 实时推送日志
3. **Results Viewer** —按帧查看定位结果 (平移/旋转误差、匹配内点数)
4. **AR Demo** —查看或在线生成 AR 图像
5. **Export** —JSON/CSV 导出、打包代码和结果

7.2 技术栈

- 后端：Python Flask，8 个 RESTful API 端点
- 实时日志：SSE (Server-Sent Events)
- 前端：原生 HTML/CSS/JS，暗色主题
- 启动：python scripts/web_ui.py，浏览器打开 <http://localhost:5000>

8 项目总结

8.1 完成工作清单

表 11: 项目完成度清单

要求	状态	说明
添加 ≥ 1 种检索方法	✓	EigenPlaces + CricaVPR (2 种)
添加 ≥ 1 种检测方法	✓	ALIKED (1 种)
添加 ≥ 1 种匹配方法	✓	LightGlue (1 种)
≥ 1 个数据集测试	✓	Cambridge + 7-Scenes (2 个)
两种 Baseline 对比	✓	BL-A (SIFT+NN), BL-B (SP+SG)
加分项	状态	说明
两个以上数据集	✓	Cambridge + 7-Scenes
两个以上方法扩展	✓	4 个新模块 (2 检索 + 1 检测 + 1 匹配)
消融实验分析	✓	8 组实验, 全模块消融
定性定量对比分析	✓	序列级分析, 检索示例, 耗时分析
自定义 AR Demo	✓	固定世界坐标立方体, 智能序列选择
完整交互界面	✓	Flask Web UI, 5 模块, 实时 SSE

8.2 核心结论

做了这么多组实验下来，有几个比较明确的结论：

1. **ALIKED + LightGlue** 是这次实验里最好的检测 + 匹配组合。精度比 SIFT+NN 高了 1.75%，速度快了差不多 3 倍。LightGlue 的自适应深度确实有用——大部分匹配对不需要跑满 9 层注意力。
2. **检索器的选择比匹配器重要得多**。换检索器导致的精度下降（6–7%）远大于换匹配器带来的提升（+1.75%）。在这个分层架构里，第一步检索如果没召回对的候选图，后面匹配再好也白搭。NetVLAD 在两个数据集上都是最好的，说明它训练的街景数据分布跟这两个数据集比较接近。

3. **7-Scenes** 的问题不在平移在旋转。平移误差 0.087m 跟 DSAC++ 在 Stairs 上的结果差不多，但旋转差了不少。Stairs 这种全是墙和楼梯的场景确实不太适合纯特征点匹配的方法。
4. **序列选择不能只看帧数**。seq3 帧数最多但定位率只有 45%，seq7 帧数不多但定位率 93%。这个发现直接改变了 AR Demo 的设计——不去手动挑序列，而是自动算每条序列的中位误差来选。
5. **三角化有收益但代价不小**。pycolmap 重三角化提了 3.5 个百分点，但要跑近 16000 对匹配和完整重建。实际部署的话得看场景是否值得这个开销。

8.3 个人体会与改进方向

这次大作业最深的感受是——把 **pipeline** 跑通需要下非常大的功夫。从刚开始所有 query 定位全错 (recall = 0%)，到最终跑出 69.68% 的高精度召回，中间踩了不少坑。记录几个花时间最多的：

- **PnP 输出与 GT 位姿的坐标系不一致**——这导致实验初期召回率一直为 0% PnP 返回的 $(\mathbf{R}, \mathbf{t})$ 满足 $\mathbf{X}_{cam} = \mathbf{R} \cdot \mathbf{X}_{world} + \mathbf{t}$ ，但 GT 存的是相机在世界坐标系中的位置和 world-to-camera 四元数。我一开始直接把 PnP 的 $\mathbf{t}$ 当成相机位置去跟 GT 比较——平移误差 20–1500m，旋转误差 90–180°，所有帧全错。正确的做法是先转成 $\mathbf{t}_{c2w} = -\mathbf{R}^T \mathbf{t}$ ，四元数也要用 $\mathbf{R}$ 而不是 $\mathbf{R}^T$ 。这个 bug 修完，recall 从 0 直接涨到 65.6%。
- **VLFeat SIFT 和 OpenCV SIFT 不兼容**：COLMAP 建 3D 模型用的是 VLFeat 版的 SIFT，但我代码里用 OpenCV SIFT。一开始在 COLMAP 记录的关键点位置用 OpenCV 重新算描述符，结果匹配全错，PnP 只能拿到 8–14 个内点。最后改成对 DB 图做 detectAndCompute，匹配时通过空间邻近 (8px 半径) 去查最近的 COLMAP 3D 点。
- **COLMAP 二进制格式解析**：数据集里 points3D.txt 是空的 (0 个点)，最后是用 model_train/ 下的二进制版本。但二进制 reader 也花了一些时间——point3D_id 是 uint64 不是 int32，cameras.bin 没有 num_params 字段，struct 格式花了一些时间调整。
- **pycolmap 三角化**：为了复现 HLoc 论文里的 SP+SG 三角化，pycolmap 建数据库的时候就遇到了 SQLite 外键约束报错、TwoViewGeometry 的 matches 数组 shape 不对 (要 $N \times 2$ 不是 $N \times 1$)、point3D_id 是 uint64 导致 int too big to convert。最终生成 178,072 个 3D 点，recall 从 66.18% 提到了 69.68%。
- **OpenMP 冲突**：Windows 上 PyTorch 和 pycolmap 都链了 OpenMP，运行时报 OMP: Error #15 crash。通过在终端命令设 KMP_DUPLICATE_LIB_OK=TRUE 解决。

分工声明

本项目为单人独立完成。我负责了以下全部工作：

- 算法调研与方案设计
- 四个扩展模块（EigenPlaces、CricaVPR、ALIKED、LightGlue）的代码集成与调试
- XRLocalization 框架修改和 Bug 修复（坐标系统一、COLMAP/NVM 读取、深度图反投影等）
- Flask Web 界面开发
- AR Demo 渲染管线实现
- 全部实验运行和数据整理
- 本报告撰写

所有扩展模块均在 XRLocalization 框架内通过继承标准接口实现。实验代码保存于 `scripts/versions/v1_cambridge/` 和 `scripts/versions/v2_7scenes/`。

9 关键代码实现

以下列出几个核心模块的关键代码，用于说明如何在 XRLocalization 框架内扩展新模块。

9.1 ALIKED 检测器适配

ALIKED 使用可变形卷积和特征金字塔做多尺度关键点检测。下面的代码把它封装成 XRLocalization 的标准检测器接口（继承 `BaseDetector`，实现 `_load_model()` 和 `detect()`）：

```
1 class ALIKEDDetector(BaseDetector):
2     """
3     XRLocalization 框架适配器 — 将 ALIKED 封装为标准检测器接口。
4     继承 BaseDetector，实现 _load_model() 和 detect() 两个方法。
5     """
6     DEFAULT_CONFIG = {
7         "max_keypoints": 5000,          # 每张图像最大关键点数
8         "detection_threshold": 0.2,    # 关键点得分阈值
9         "device": "cuda",
10    }
11
12    def _load_model(self):
13        """加载 ALIKED 预训练模型"""
14        from lightglue import ALIKED
15        self.model = ALIKED(
16            max_num_keypoints=self.config.get("max_keypoints", 5000),
```

```

17         detection_threshold=self.config.get("detection_threshold",
18             0.2),
19     ).eval().to(self.device)
20
21 def detect(self, image: np.ndarray):
22     """
23     对输入图像提取关键点和描述符。
24     Returns: keypoints(N,2), descriptors(N,D), scores(N,)
25     """
26     # uint8 (H,W,C) → float32 (1,C,H,W) [0,1]
27     image_t = torch.from_numpy(
28         image.transpose(2, 0, 1)).float().unsqueeze(0) / 255.0
29     image_t = image_t.to(self.device)
30
31     with torch.no_grad():
32         feats = self.model.extract(image_t)
33
34     keypoints = feats["keypoints"][0].cpu().numpy().astype(np.float32)
35
36     descriptors = feats["descriptors"][0].cpu().numpy().astype(np.
37         float32)
38
39     scores = feats["keypoint_scores"][0].cpu().numpy().astype(np.
40         float32)
41
42     # L2 归一化描述符 → 余弦相似度等价于内积
43     norms = np.linalg.norm(descriptors, axis=1, keepdims=True)
44     descriptors = descriptors / np.clip(norms, a_min=1e-12, a_max=
45         None)
46
47     scores = np.clip(scores, 0.0, 1.0)
48     return keypoints, descriptors, scores

```

Listing 1: ALIKED 检测器适配代码

9.2 LightGlue 匹配器适配

LightGlue 的核心是自适应深度——根据两张图的匹配难度自动决定用多少层注意力。以下代码展示如何把它封装成 XRLocalization 的标准匹配器接口：

```

1 class LightGlueMatcher(BaseMatcher):
2     """
3     XRLocalization 框架适配器 — 将 LightGlue 封装为标准匹配器接口。
4     支持 superpoint / aliked / disk / sift 多种特征类型。
5     """
6     DEFAULT_CONFIG = {

```

```

7         "features": "aliked",
8         "depth_confidence": 0.95,          # 自适应深度置信度
9         "filter_threshold": 0.1,          # 匹配置信度过滤
10        "device": "cuda",
11    }
12
13    def match(self, query_data, db_data):
14        """
15        Query 与 DB 图像关键点之间建立匹配对应关系。
16        Returns: q_idx(M,), db_idx(M,), conf(M,)
17        """
18        # 构建 LightGlue 输入字典 (batch 维度=1)
19        data = {
20            "image0": {
21                "keypoints": torch.from_numpy(
22                    kpts0[np.newaxis]).float().to(self.device),
23                "descriptors": torch.from_numpy(
24                    desc0[np.newaxis]).float().to(self.device),
25            },
26            "image1": { /* 同上结构 */ },
27        }
28
29        # 前向推理 — 自适应深度在此自动决策
30        with torch.no_grad():
31            pred = self.model(data)
32            matches = pred["matches0"][0].cpu().numpy()
33
34            # matches[i] = j → Query 第 i 点匹配到 DB 第 j 点
35            # matches[i] = -1 → 未匹配, 过滤
36            valid = matches > -1
37            q_idx = np.where(valid)[0]
38            db_idx = matches[valid]
39            conf = pred["matching_scores0"][0].cpu().numpy()[valid]
40        return q_idx, db_idx, conf

```

Listing 2: LightGlue 匹配器适配代码

9.3 AR Demo 透视投影与渲染

从固定世界坐标的 3D 立方体, 通过预测的相机位姿投影到 2D 图像上:

```

1 def project_points(pts3d_world, t_c2w, q_w2c, K):
2     """
3     透视投影:  $p = K \cdot (R_{w2c} \cdot P_w + t_{cam})$ 

```

```

4     其中  $t_{cam} = -R_{w2c} \cdot t_{c2w}$  (相机中心到光心的偏移)
5     """
6     R_w2c = quat_to_rotmat(q_w2c)
7     pts_cam = (R_w2c @ pts3d_world.T).T + (-R_w2c @ t_c2w)
8     pts_img = (K @ pts_cam.T).T
9     depth = pts_img[:, 2] # Z = 深度
10    pts2d = pts_img[:, :2] / depth[:, np.newaxis] # 透视除法
11    return pts2d, depth
12
13 def draw_cube(image, verts2d, edges, faces, depth, color, alpha,
14               line_width):
15     """半透明立方体渲染: 面片填充 + 边缘线框"""
16     overlay = image.copy()
17     # 面片填充 — 仅渲染深度 > 0 的面
18     for tri in faces:
19         if all(depth[idx] > 0 for idx in tri):
20             pts = [(int(verts2d[idx][0]), int(verts2d[idx][1]))
21                   for idx in tri]
22             cv2.fillPoly(overlay, [np.array(pts)], color_bgr)
23     # 半透明混合
24     cv2.addWeighted(overlay, alpha, image, 1 - alpha, 0, image)
25     # 边缘线框
26     for i, j in edges:
27         if depth[i] > 0 and depth[j] > 0:
28             cv2.line(image, tuple(verts2d[i].astype(int)),
29                     tuple(verts2d[j].astype(int)), color_bgr, line_width)
30
31     return image

```

Listing 3: 透视投影与立方体渲染

9.4 7-Scenes 深度图反投影

7-Scenes 没有 COLMAP 模型, 需要用深度图把 2D 关键点反投影到 3D:

```

1 def _depth_unproject(keypoints, depth_map, K, R_c2w, t_c2w):
2     """
3     利用深度图将 2D 关键点反投影为 3D 世界坐标。
4
5      $X_{cam} = [(u-cx)*z/fx, (v-cy)*z/fy, z]$ 
6      $X_{world} = R_{c2w} @ X_{cam} + t_{c2w}$ 
7     """
8     fx, fy = K[0, 0], K[1, 1]
9     cx, cy = K[0, 2], K[1, 2]

```

```

10
11 pts3d_world = []
12 for (x, y) in keypoints.astype(int):
13     z = depth_map[y, x] / 1000.0      # mm → m
14     if z <= 0.01 or z > 10.0:        # 过滤无效深度
15         continue
16     X_cam = np.array([
17         (x - cx) * z / fx,
18         (y - cy) * z / fy,
19         z
20     ])
21     X_world = R_c2w @ X_cam + t_c2w
22     pts3d_world.append(X_world)
23
24 return np.array(pts3d_world)

```

Listing 4: 深度图反投影: 2D → 3D

9.5 智能序列选择

不是手动指定用哪个序列，而是读 per_frame.csv 算每条序列的中位平移误差，自动选最好的：

```

1 def sample_frames(items, limit, per_frame_errors=None):
2     """
3     选择定位质量最优的序列（而非最长序列）。
4
5     序列质量评分：(0, median_t_err, -len) → 取最小值
6     - 有 per_frame 数据 → (0, 中位平移误差, -帧数)：优先低误差
7     - 无 per_frame 数据 → (1, 0, -帧数)：回退到最长序列
8     """
9     def _seq_score(seq_name):
10         if per_frame_errors and seq_name in per_frame_errors:
11             errs = per_frame_errors[seq_name]
12             if errs:
13                 return (0, np.median(errs), -len(seqs[seq_name]))
14             return (1, 0, -len(seqs[seq_name]))
15
16     best_seq = min(seqs.keys(), key=_seq_score)
17     # 从最优序列中按帧号均匀采样，生成连续视频
18     ...

```

Listing 5: 智能序列选择

参考文献

- [1] P.-E. Sarlin, C. Cadena, R. Siegwart, and M. Dymczyk. *From Coarse to Fine: Robust Hierarchical Localization at Large Scale*. In CVPR, 2019.
- [2] T. Sattler, B. Leibe, and L. Kobbelt. *Efficient & Effective Prioritized Matching for Large-Scale Image-Based Localization*. In PAMI, 2017.
- [3] A. Kendall, M. Grimes, and R. Cipolla. *PoseNet: A Convolutional Network for Real-Time 6-DOF Camera Relocalization*. In ICCV, 2015.
- [4] E. Brachmann and C. Rother. *Learning Less is More —6D Camera Localization via 3D Surface Regression*. In CVPR, 2018.
- [5] Z. Huang et al. *VS-Net: Voting with Segmentation for Visual Localization*. In CVPR, 2022.
- [6] R. Arandjelović, P. Gronat, A. Torii, T. Pajdla, and J. Sivic. *NetVLAD: CNN Architecture for Weakly Supervised Place Recognition*. In PAMI, 2017.
- [7] G. Berton, C. Masone, and B. Caputo. *EigenPlaces: Training Viewpoint Robust Models for Visual Place Recognition*. In ICCV, 2023.
- [8] Z. Fan et al. *CricaVPR: Cross-Image Correlation-Aware Representation Learning for Visual Place Recognition*. In CVPR, 2024.
- [9] X. Wang et al. *ALIKED: A Lighter Keypoint and Descriptor Extraction Network via Deformable Transformation*. In T-IM, 2023.
- [10] P. Lindenberger, P.-E. Sarlin, and M. Pollefeys. *LightGlue: Local Feature Matching at Light Speed*. In ICCV, 2023.
- [11] D. DeTone, T. Malisiewicz, and A. Rabinovich. *SuperPoint: Self-Supervised Interest Point Detection and Description*. In CVPR Workshops, 2018.
- [12] P.-E. Sarlin, D. DeTone, T. Malisiewicz, and A. Rabinovich. *SuperGlue: Learning Feature Matching with Graph Neural Networks*. In CVPR, 2020.
- [13] J. L. Schönberger and J.-M. Frahm. *Structure-from-Motion Revisited*. In CVPR, 2016.
- [14] J. Shotton et al. *Scene Coordinate Regression Forests for Camera Relocalization in RGB-D Images*. In CVPR, 2013.